Mini-TP — ReplicaSets, mise à jour et retour arrière

Un Deployment Nginx dans un cluster minikube de trois nœuds

Objectif

Vous allez créer un **Deployment**, observer les **ReplicaSets** que Kubernetes génère pour lui, faire une mise à jour progressive (*rolling update*), revenir en arrière (*rollback*), puis repartir vers une nouvelle version. Durée : 15 minutes environ.

Temps	Image	Ce que vous observez
1	`nginx:1.23`	un Deployment, un ReplicaSet, 3 pods
2	`nginx:1.25`	un deuxième ReplicaSet monte, le premier descend à 0
3	retour à `nginx:1.23`	l'ancien ReplicaSet se réveille, aucun nouveau n'est créé
4	`nginx:1.27`	un troisième ReplicaSet, les deux anciens restent à 0

Prérequis

- **Docker** démarré, avec au moins **8 Go** de mémoire.
- **minikube** et **kubectl**.
- Un terminal **bash** ou **zsh** : Linux, macOS, WSL ou Git Bash. Les commandes de ce TP utilisent des guillemets simples et des `\` en fin de ligne : elles ne fonctionnent pas telles quelles dans PowerShell ou CMD.

Conventions

- Vous travaillez avec **deux terminaux** : le **terminal principal**, où vous tapez les commandes, et le **terminal d'observation**, qui affiche l'état du cluster en continu.
- Les noms des ReplicaSets et des pods se terminent par un suffixe aléatoire (`mon-site-6b8f9c7d4f…`). Dans les résultats attendus, ils sont notés `mon-site-aaaaaaaaaa`, `mon-site-bbbbbbbbbb`, `mon-site-cccccccccc` pour que vous puissiez les suivre. Les durées et les numéros de version exacts varient aussi.

Étape 1 — Vérifier vos outils

1. Vérifiez que Docker répond (cette commande échoue si Docker n'est pas démarré) :

```
docker version --format '{{.Server.Version}}'
```

Résultat attendu :

```
28.x.x
```

2. Vérifiez les autres outils :

```
minikube version
kubectl version --client
```

Résultat attendu :

```
minikube version: v1.39.x
...
Client Version: v1.3x.x
```

Contrôle : chaque commande affiche une version, sans erreur.

Étape 2 — Créer un cluster de trois nœuds

Le cluster de ce TP s'appelle `tp-rs`. Il compte **un nœud control plane et deux nœuds de travail** : vous verrez ainsi les pods se répartir sur plusieurs machines.

1. Si votre cluster `formation` est démarré, arrêtez-le pour libérer de la mémoire (sinon, passez à l'action suivante) :

   ```
   minikube stop -p formation
   ```

   Résultat attendu :

   ```
   Stopping node "formation"  ...
   1 node stopped.
   ```

   Contrôle : le cluster `formation` n'est pas supprimé : vous le redémarrerez à la fin du TP.

2. Créez le cluster `tp-rs` avec trois nœuds de 2 processeurs et 2 Go de mémoire chacun :

   ```
   minikube start -p tp-rs --driver=docker --nodes=3 --cpus=2 --memory=2g
   ```

   Résultat attendu :

   ```
   [tp-rs] minikube v1.39.x ...
   Using the docker driver based on user configuration
   Starting "tp-rs" primary control-plane node in "tp-rs" cluster
   ...
   Starting "tp-rs-m02" worker node in "tp-rs" cluster
   ...
   Starting "tp-rs-m03" worker node in "tp-rs" cluster
   ...
   Done! kubectl is now configured to use "tp-rs" cluster and "default" namespace by default
   ```

   Contrôle : trois lignes `Starting`, une par nœud, et la ligne finale `Done!`. Comptez deux à trois minutes.

3. Vérifiez que kubectl s'adresse bien à ce cluster :

   ```
   kubectl config current-context
   ```

   Résultat attendu :

   ```
   tp-rs
   ```

   Contrôle : exactement `tp-rs`. Sinon, basculez dessus avec `kubectl config use-context tp-rs`.

4. Listez les nœuds :

   ```
   kubectl get nodes
   ```

   Résultat attendu :

   ```
   NAME                 STATUS   ROLES           AGE   VERSION
   tp-rs                Ready    control-plane   1m    v1.3x.x
   tp-rs-m02            Ready    <none>          1m    v1.3x.x
   tp-rs-m03            Ready    <none>          1m    v1.3x.x
   ```

   Contrôle : 3 nœuds `Ready`. Si un nœud est encore `NotReady`, attendez quelques secondes et relancez la commande.

Étape 3 — Charger les images Nginx dans le cluster

Docker Hub limite le nombre de téléchargements anonymes. Si toute une salle télécharge les mêmes images en même temps, des pods peuvent rester bloqués en `ImagePullBackOff` au milieu de la mise à jour. Vous allez donc télécharger chaque image **une seule fois** sur votre poste, puis la copier dans les nœuds du cluster.

1. Téléchargez les trois versions et chargez-les dans le cluster :

```
for tag in 1.23 1.25 1.27; do
  docker pull nginx:$tag
  minikube image load nginx:$tag -p tp-rs
done
```

Contrôle : chaque `docker pull` se termine par `Status: Downloaded newer image for nginx:...` (ou `Image is up to date`). `minikube image load` n'affiche rien quand tout va bien.

2. Vérifiez que les images sont présentes sur un nœud de travail :

```
minikube ssh -p tp-rs -n tp-rs-m02 -- sudo crictl images | grep nginx
```

Résultat attendu :

```
docker.io/library/nginx   1.23   a7be6198544f0   ...
docker.io/library/nginx   1.25   9d6767b714bf1   ...
docker.io/library/nginx   1.27   1e5f3c5b981a2   ...
```

Contrôle : les trois versions. Comme leurs tags sont précis (et non `latest`), Kubernetes utilisera ces images déjà présentes sans retourner sur Docker Hub.

Étape 4 — Créer un namespace et comprendre le principe

1. Créez le namespace du TP :

```
kubectl create namespace tp-rs
```

Résultat attendu :

```
namespace/tp-rs created
```

Avant de manipuler, retenez la chaîne de commandement :

- un **Deployment** ne crée pas de pods lui-même : il pilote un ou plusieurs **ReplicaSets** ;
- chaque ReplicaSet correspond à **une version précise du modèle de pod** (le *template* : image, variables, ports…) et maintient le bon nombre de pods ;
- quand vous changez l'image du Deployment, Kubernetes crée un **nouveau** ReplicaSet, le fait monter progressivement pendant que l'ancien descend à 0 ;
- l'ancien ReplicaSet n'est **pas supprimé** : il reste à 0 pour permettre un retour arrière.

Étape 5 — Ouvrir le terminal d'observation

1. Ouvrez un **deuxième terminal** et vérifiez qu'il vise le même cluster :

```
kubectl config current-context
```

Résultat attendu :

```
tp-rs
```

2. Lancez l'affichage en continu du Deployment, de ses ReplicaSets et de ses pods :

```
watch -n 1 "kubectl get deploy,rs,pods -n tp-rs -o wide"
```

Résultat attendu :

```
Every 1.0s: kubectl get deploy,rs,pods -n tp-rs -o wide

No resources found in tp-rs namespace.
```

Contrôle : l'écran se rafraîchit chaque seconde. Il est vide pour l'instant : c'est normal. **Laissez ce terminal ouvert et visible** pendant tout le TP.

> **Attention :** `watch` n'existe pas par défaut sous macOS (installez-le avec `brew install watch`) ni dans Git Bash. À la place, utilisez `kubectl get rs -n tp-rs -w` : au lieu de rafraîchir l'écran, il ajoute une ligne à chaque changement.

Étape 6 — Déployer la version 1 : nginx:1.23

Toutes les commandes suivantes se tapent dans le **terminal principal**.

1. Créez le Deployment `mon-site` avec 3 copies de `nginx:1.23`, puis attendez qu'il soit prêt :

```
kubectl create deployment mon-site --image=nginx:1.23 --replicas=3 -n tp-rs
kubectl rollout status deployment/mon-site -n tp-rs
```

Résultat attendu :

```
deployment.apps/mon-site created
Waiting for deployment "mon-site" rollout to finish: 0 of 3 updated replicas are available...
deployment "mon-site" successfully rolled out
```

2. Listez les trois niveaux d'objets :

```
kubectl get deploy,rs,pods -n tp-rs
```

Résultat attendu :

```
NAME                         READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/mon-site     3/3     3            3           30s

NAME                                    DESIRED   CURRENT   READY   AGE
replicaset.apps/mon-site-aaaaaaaaaa     3         3         3       30s

NAME                                READY   STATUS    RESTARTS   AGE
pod/mon-site-aaaaaaaaaa-x7k2p       1/1     Running   0          30s
pod/mon-site-aaaaaaaaaa-m4q9z       1/1     Running   0          30s
pod/mon-site-aaaaaaaaaa-t8w3n       1/1     Running   0          30s
```

Contrôle : 1 Deployment, 1 ReplicaSet, 3 pods. Le nom du ReplicaSet commence par celui du Deployment ; le nom de chaque pod commence par celui du ReplicaSet. C'est la chaîne de commandement, visible dans les noms.

3. Regardez sur quels nœuds les pods ont été placés :

```
kubectl get pods -n tp-rs -o wide
```

Résultat attendu :

```
NAME                          READY   STATUS    ...   NODE
mon-site-aaaaaaaaaa-x7k2p     1/1     Running   ...   tp-rs-m02
mon-site-aaaaaaaaaa-m4q9z     1/1     Running   ...   tp-rs-m03
mon-site-aaaaaaaaaa-t8w3n     1/1     Running   ...   tp-rs
```

Contrôle : les pods sont répartis sur plusieurs nœuds. Avec minikube, le control plane `tp-rs` accepte lui aussi des applications ordinaires.

4. Préparez un affichage des ReplicaSets avec l'image portée par chacun. La variable `COLS` décrit les colonnes voulues ; elle ne vit que dans ce terminal :

```
COLS='NAME:.metadata.name,DESIRED:.spec.replicas,CURRENT:.status.replicas,'
COLS+='READY:.status.readyReplicas,IMAGE:.spec.template.spec.containers[0].image'
```

5. Utilisez-la :

```
kubectl get rs -n tp-rs -o custom-columns="$COLS"
```

Résultat attendu :

```
NAME                  DESIRED   CURRENT   READY   IMAGE
mon-site-aaaaaaaaaa   3         3         3       nginx:1.23
```

Contrôle : un seul ReplicaSet, qui maintient 3 pods en `nginx:1.23`. Si la commande affiche une erreur sur les colonnes, redéfinissez `COLS` (action 4) : vous êtes sans doute dans un autre terminal.

Étape 7 — Rendre la mise à jour observable

Sur un cluster local, une mise à jour de 3 pods Nginx dure à peine quelques secondes. Vous allez la rendre plus lisible en réglant deux paramètres du Deployment :

- `minReadySeconds: 5` : un nouveau pod n'est compté **disponible** que 5 secondes après être devenu prêt. C'est ce réglage qui ralentit la mise à jour ;
- `maxSurge: 1` et `maxUnavailable: 1` : pendant la mise à jour, il peut exister **un pod de plus** que demandé (4 au maximum) et **un pod de moins** disponible (2 au minimum). Vous verrez ainsi les deux ReplicaSets à 2 pods en même temps.

1. Appliquez ces réglages (copiez le bloc entier : le JSON tient sur plusieurs lignes entre les guillemets simples) :

```
kubectl patch deployment/mon-site -n tp-rs -p '{
  "spec": {
    "minReadySeconds": 5,
    "strategy": { "rollingUpdate": { "maxSurge": 1, "maxUnavailable": 1 } }
  }
}'
```

Résultat attendu :

```
deployment.apps/mon-site patched
```

2. Vérifiez qu'il n'y a toujours qu'un seul ReplicaSet :

```
kubectl get rs -n tp-rs -o custom-columns="$COLS"
```

Résultat attendu :

```
NAME                  DESIRED   CURRENT   READY   IMAGE
mon-site-aaaaaaaaaa   3         3         3       nginx:1.23
```

Contrôle : toujours **un seul** ReplicaSet. Ces réglages concernent le Deployment, pas le modèle de pod : ils ne créent donc pas de nouvelle version.

Étape 8 — Mettre à jour vers la version 2 : nginx:1.25

1. Changez l'image, puis **regardez immédiatement le terminal d'observation** :

```
kubectl set image deployment/mon-site nginx=nginx:1.25 -n tp-rs
```

Résultat attendu :

```
deployment.apps/mon-site image updated
```

Dans `nginx=nginx:1.25`, la partie gauche est le **nom du conteneur** à modifier, la partie droite la nouvelle image. `kubectl create deployment` a nommé le conteneur d'après l'image : il s'appelle donc `nginx`.

2. Dans le terminal d'observation, repérez le passage de relais entre les deux ReplicaSets. Pendant quelques secondes, vous voyez par exemple :

   Résultat attendu :

```
NAME                                  DESIRED   CURRENT   READY   AGE
replicaset.apps/mon-site-aaaaaaaaaa   2         2         2       4m
replicaset.apps/mon-site-bbbbbbbbbb   2         2         1       10s
```

Puis, à la fin :

```
NAME                                  DESIRED   CURRENT   READY   AGE
replicaset.apps/mon-site-aaaaaaaaaa   0         0         0       5m
replicaset.apps/mon-site-bbbbbbbbbb   3         3         3       30s
```

   Contrôle : deux ReplicaSets : l'ancien descend, le nouveau monte. Les chiffres intermédiaires exacts dépendent du moment où l'écran se rafraîchit. Dans la partie pods, des pods `mon-site-aaaaaaaaaa-…` passent en `Terminating` pendant que des pods `mon-site-bbbbbbbbbb-…` démarrent.

3. Dans le terminal principal, attendez la fin de la mise à jour :

```
kubectl rollout status deployment/mon-site -n tp-rs
```

   Résultat attendu :

```
deployment "mon-site" successfully rolled out
```

4. Affichez les ReplicaSets avec leur image :

```
kubectl get rs -n tp-rs -o custom-columns="$COLS"
```

   Résultat attendu :

```
NAME                  DESIRED   CURRENT   READY    IMAGE
mon-site-aaaaaaaaaa   0         0         <none>   nginx:1.23
mon-site-bbbbbbbbbb   3         3         3        nginx:1.25
```

   Contrôle : l'ancien ReplicaSet (`nginx:1.23`) est à 0 mais **toujours là** ; le nouveau (`nginx:1.25`) porte les 3 pods. `<none>` signifie simplement « aucun pod prêt » : Kubernetes n'enregistre pas ce compteur quand il vaut 0. L'ordre des lignes peut être inversé chez vous : repérez les ReplicaSets par leur image.

Étape 9 — Consulter l'historique des versions

1. Affichez l'historique du Deployment :

```
kubectl rollout history deployment/mon-site -n tp-rs
```

   Résultat attendu :

```
deployment.apps/mon-site
REVISION  CHANGE-CAUSE
1         <none>
2         <none>
```

   Contrôle : deux **révisions**, une par ReplicaSet. La colonne `CHANGE-CAUSE` est vide (`<none>`) car vous n'avez pas encore décrit vos changements : vous le ferez à l'étape 11.

2. Affichez le détail de chaque révision :

```
kubectl rollout history deployment/mon-site -n tp-rs --revision=1
kubectl rollout history deployment/mon-site -n tp-rs --revision=2
```

Résultat attendu :

```
deployment.apps/mon-site with revision #1
Pod Template:
  ...
  Containers:
   nginx:
    Image:      nginx:1.23
...
deployment.apps/mon-site with revision #2
...
    Image:      nginx:1.25
```

Contrôle : la révision 1 porte `nginx:1.23`, la révision 2 porte `nginx:1.25`.

Étape 10 — Revenir en arrière

Imaginez que la version `nginx:1.25` pose problème en production. Vous revenez à la version précédente.

1. Lancez le retour arrière, puis regardez le terminal d'observation :

```
kubectl rollout undo deployment/mon-site -n tp-rs
```

Résultat attendu :

```
deployment.apps/mon-site rolled back
```

2. Dans le terminal d'observation, constatez que c'est **l'ancien** ReplicaSet qui se réveille. Par exemple :

Résultat attendu :

```
NAME                                  DESIRED   CURRENT   READY   AGE
replicaset.apps/mon-site-aaaaaaaaaa   2         2         1       8m
replicaset.apps/mon-site-bbbbbbbbbb   2         2         2       3m
```

Contrôle : toujours **deux** ReplicaSets, pas trois : `mon-site-aaaaaaaaaa` remonte.

3. Dans le terminal principal, attendez la fin, puis affichez les ReplicaSets :

```
kubectl rollout status deployment/mon-site -n tp-rs
kubectl get rs -n tp-rs -o custom-columns="$COLS"
```

Résultat attendu :

```
deployment "mon-site" successfully rolled out
NAME                  DESIRED   CURRENT   READY    IMAGE
mon-site-aaaaaaaaaa   3         3         3        nginx:1.23
mon-site-bbbbbbbbbb   0         0         <none>   nginx:1.25
```

Contrôle : le ReplicaSet `nginx:1.23` est revenu à 3 pods. Kubernetes n'a pas eu besoin d'en créer un nouveau : le modèle de pod `nginx:1.23` existait déjà dans l'ancien ReplicaSet, il l'a simplement remonté.

4. Affichez à nouveau l'historique :

```
kubectl rollout history deployment/mon-site -n tp-rs
```

Résultat attendu :

```
deployment.apps/mon-site
REVISION  CHANGE-CAUSE
2         <none>
3         <none>
```

Contrôle : la révision 1 a **disparu** et une révision 3 est apparue. Le modèle `nginx:1.23` a été promu au rang de révision la plus récente : il ne peut pas figurer deux fois dans l'historique.

À retenir : un numéro de révision n'est **pas** un numéro de version de votre application. C'est un compteur d'événements : un retour arrière crée une nouvelle révision. Pour savoir ce que contient une révision, consultez son détail (`--revision=N`).

Étape 11 — Repartir vers la version 3 : nginx:1.27

Le problème est corrigé : vous déployez `nginx:1.27`. Cette fois, vous décrivez le changement pour qu'il apparaisse dans l'historique.

1. Changez l'image, puis notez la raison du changement dans l'annotation `kubernetes.io/change-cause` :

```
kubectl set image deployment/mon-site nginx=nginx:1.27 -n tp-rs
kubectl annotate deployment/mon-site -n tp-rs \
  kubernetes.io/change-cause="passage en nginx 1.27"
```

Résultat attendu :

```
deployment.apps/mon-site image updated
deployment.apps/mon-site annotated
```

2. Attendez la fin de la mise à jour, puis affichez les ReplicaSets :

```
kubectl rollout status deployment/mon-site -n tp-rs
kubectl get rs -n tp-rs -o custom-columns="$COLS"
```

Résultat attendu :

```
deployment "mon-site" successfully rolled out
NAME                  DESIRED   CURRENT   READY    IMAGE
mon-site-aaaaaaaaaa   0         0         <none>   nginx:1.23
mon-site-bbbbbbbbbb   0         0         <none>   nginx:1.25
mon-site-cccccccccc   3         3         3        nginx:1.27
```

Contrôle : trois ReplicaSets : `nginx:1.27` porte les 3 pods, les deux anciens restent à 0, prêts pour un éventuel retour arrière.

3. Affichez l'historique :

```
kubectl rollout history deployment/mon-site -n tp-rs
```

Résultat attendu :

```
deployment.apps/mon-site
REVISION  CHANGE-CAUSE
2         <none>
3         <none>
4         passage en nginx 1.27
```

Contrôle : la révision 4 porte votre description. Pensez à annoter chaque mise à jour : c'est la seule trace lisible de ce qui a changé.

4. Vérifiez l'image réellement utilisée par les pods :

```
PCOLS='NAME:.metadata.name,READY:.status.containerStatuses[0].ready,'
PCOLS+='IMAGE:.spec.containers[0].image'
kubectl get pods -n tp-rs -o custom-columns="$PCOLS"
```

Résultat attendu :

```
NAME                         READY   IMAGE
mon-site-ccccccccc-h2v6r   true    nginx:1.27
mon-site-ccccccccc-p9d4s   true    nginx:1.27
mon-site-ccccccccc-z5b8k   true    nginx:1.27
```

Contrôle : 3 pods, tous en `nginx:1.27`, tous prêts (`true`), tous issus du ReplicaSet `mon-site-ccccccccc`.

Étape 12 — Limiter l'historique

Les anciens ReplicaSets à 0 ne disparaissent pas d'eux-mêmes. Le champ `revisionHistoryLimit` du Deployment fixe combien d'entre eux sont conservés. Un ancien ReplicaSet disparaît quand cette limite est dépassée, quand vous supprimez le Deployment, ou quand vous le supprimez à la main.

1. Affichez la limite actuelle :

```
kubectl get deployment mon-site -n tp-rs -o jsonpath='{.spec.revisionHistoryLimit}{"\n"}'
```

Résultat attendu :

```
10
```

Contrôle : `10` : la valeur par défaut.

2. Réduisez-la à 2, puis comptez les ReplicaSets :

```
kubectl patch deployment/mon-site -n tp-rs -p '{"spec":{"revisionHistoryLimit":2}}'
kubectl get rs -n tp-rs
```

Résultat attendu :

```
deployment.apps/mon-site patched
NAME                  DESIRED   CURRENT   READY   AGE
mon-site-aaaaaaaaaa   0         0         0       20m
mon-site-bbbbbbbbbb   0         0         0       15m
mon-site-cccccccccc   3         3         3       3m
```

Contrôle : toujours 3 ReplicaSets : vous n'avez que 2 anciens ReplicaSets, la limite n'est pas dépassée.

3. Passez maintenant la limite à 0, puis recomptez :

```
kubectl patch deployment/mon-site -n tp-rs -p '{"spec":{"revisionHistoryLimit":0}}'
kubectl get rs -n tp-rs
```

Résultat attendu :

```
deployment.apps/mon-site patched
NAME                  DESIRED   CURRENT   READY   AGE
mon-site-cccccccccc   3         3         3       4m
```

Contrôle : les deux anciens ReplicaSets ont été **supprimés**. Si vous les voyez encore, relancez `kubectl get rs -n tp-rs` après quelques secondes.

4. Tentez un retour arrière :

```
kubectl rollout undo deployment/mon-site -n tp-rs
```

Résultat attendu :

```
error: no rollout history found for deployment "mon-site"
```

Contrôle : le retour arrière est **impossible** : il n'existe plus aucune version précédente. L'application continue de tourner normalement en `nginx:1.27`.

Attention : réduire `revisionHistoryLimit` réduit d'autant vos possibilités de retour arrière. Ne le mettez jamais à 0 sur une application réelle.

Étape 13 — Nettoyer

1. Dans le terminal d'observation, arrêtez l'affichage avec `Ctrl+C`.
2. Dans le terminal principal, supprimez le cluster du TP :

```
minikube delete -p tp-rs
```

Résultat attendu :

```
Deleting "tp-rs" in docker ...
...
Removed all traces of the "tp-rs" cluster.
```

Contrôle : `minikube profile list` ne mentionne plus `tp-rs`.

3. Si vous aviez arrêté votre cluster `formation` à l'étape 2, redémarrez-le :

```
minikube start -p formation
kubectl config current-context
```

Résultat attendu :

```
...
Done! kubectl is now configured to use "formation" cluster and "default" namespace by default
formation
```

Pour ne supprimer que les objets du TP en gardant le cluster, `kubectl delete namespace tp-rs` aurait suffi : supprimer un namespace supprime tout ce qu'il contient.

Mémo des commandes

Pour…	Commande
créer un Deployment	`kubectl create deployment mon-site --image=nginx:1.23 --replicas=3 -n tp-rs`
voir les trois niveaux	`kubectl get deploy,rs,pods -n tp-rs`
observer en continu	`watch -n 1 "kubectl get deploy,rs,pods -n tp-rs -o wide"`
changer l'image	`kubectl set image deployment/mon-site nginx=nginx:1.25 -n tp-rs`
décrire le changement	`kubectl annotate deployment/mon-site -n tp-rs kubernetes.io/change-cause="…"`
suivre une mise à jour	`kubectl rollout status deployment/mon-site -n tp-rs`
voir l'historique	`kubectl rollout history deployment/mon-site -n tp-rs`
voir une révision	`kubectl rollout history deployment/mon-site -n tp-rs --revision=2`
revenir en arrière	`kubectl rollout undo deployment/mon-site -n tp-rs`
supprimer le cluster	`minikube delete -p tp-rs`

À retenir

À retenir : un **Deployment** décrit l'état voulu ; il le réalise à travers des **ReplicaSets**.

un ReplicaSet = **une version du modèle de pod**. Changer l'image crée un nouveau ReplicaSet.

une mise à jour progressive fait **monter** le nouveau ReplicaSet et **descendre** l'ancien à 0.

un retour arrière **réveille** l'ancien ReplicaSet au lieu d'en créer un nouveau, et crée une nouvelle révision.

les anciens ReplicaSets à 0 sont gardés pour permettre le retour arrière, dans la limite de `revisionHistoryLimit` (10 par défaut).

une version sortie de l'historique ne peut plus être restaurée par `kubectl rollout undo`.